Deep Residual Learning for Image Recognition

He, Zhang, Ren, Sun — Microsoft Research, 2015

arXiv :1512.03385

ILSVRC & COCO 2015

1ère place : classification ImageNet, détection ImageNet/COCO,
localisation ImageNet, segmentation COCO

Réalisé par : Zaroual loubna

Encadré par : P.Mohamed Benaddy

26 septembre 2026

Table des matières

1 Contexte et problématique **2**

2 Contribution principale : l'apprentissage résiduel **2**
2.1 Justification . . . 2
2.2 Formulation . . . 2

3 Architectures étudiées **3**
3.1 Blocs de base et blocs « bottleneck » . . . 3
3.2 Détails d'entraînement (ImageNet) . . . 3

4 Résultats expérimentaux **3**
4.1 ImageNet — réseaux plain vs résiduels (top-1, 10-crop) . . . 3
4.2 Meilleurs résultats single-model sur ImageNet (top-5) . . . 3
4.3 CIFAR-10 : de 20 à 1202 couches . . . 4
4.4 Détection d'objets et segmentation (transfert) . . . 4

5 Analyse et discussion **4**

6 Conclusion **4**

7 Démonstration complémentaire (notebook) **4**

8 Références **5**

1 Contexte et problématique

Les auteurs partent d'un constat empirique surprenant : sur ImageNet comme sur CIFAR-10, un réseau convolutif « *plain* » (empilement simple de couches, sans mécanisme particulier) de 56 couches obtient une erreur d'entraînement et de test plus élevée qu'un réseau plain de 20 couches — alors que le réseau profond a strictement plus de capacité de représentation.

Ce phénomène, appelé **problème de dégradation**, présente deux caractéristiques importantes :

- ce n'est pas de l'*overfitting* : l'erreur d'entraînement elle-même augmente avec la profondeur ;
- ce n'est pas non plus expliqué par la disparition de gradient (*vanishing gradient*), déjà atténuée par l'initialisation normalisée et la *Batch Normalization*.

Argument théorique. Si l'on prend un réseau peu profond et qu'on lui ajoute des couches supplémentaires configurées comme des applications identité, le réseau plus profond ne devrait *jamais* faire pire que le réseau peu profond, puisqu'une solution par construction existe. Le fait que les solveurs de type SGD ne parviennent pas à trouver une solution au moins aussi bonne indique une **difficulté d'optimisation**, et non une limite de capacité du modèle.

2 Contribution principale : l'apprentissage résiduel

Plutôt que de faire apprendre directement à un empilement de couches la fonction cible $H(x)$, les auteurs proposent de lui faire apprendre la **fonction résiduelle** :

$$F(x) := H(x) - x$$

la sortie du bloc étant alors reconstruite comme :

$$y = F(x) + x$$

grâce à une **connexion de raccourci** (*shortcut connection*) qui effectue une simple addition élément par élément entre l'entrée du bloc et sa sortie.

2.1 Justification

- Si l'application identité est optimale pour un bloc donné, il est beaucoup plus facile pour le solveur de pousser les poids de F vers zéro que d'apprendre une identité exacte au travers de plusieurs couches non linéaires ;
- Les raccourcis identité n'ajoutent ni paramètre ni coût de calcul significatif (une simple addition), ce qui permet de comparer un réseau plain et son équivalent résiduel à nombre de paramètres, profondeur et coût strictement identiques ;
- Quand les dimensions changent (doublement du nombre de filtres), deux options sont comparées :
  (A) raccourci identité complété par du *zero-padding* (aucun paramètre supplémentaire) ;
  (B) raccourci de projection via une convolution 1×1.
  Les deux fonctionnent bien ; l'option B est légèrement meilleure mais plus coûteuse.

2.2 Formulation

Pour un bloc à deux couches :

$$F(x, \{W_i\}) = W_2 \cdot \sigma(W_1 \cdot x)$$

où σ est la fonction `ReLU` (biais omis). La non-linéarité finale est appliquée après l'addition :

$$y = \sigma\big(F(x) + x\big)$$

3 Architectures étudiées

3.1 Blocs de base et blocs « bottleneck »

Pour ImageNet, les auteurs déclinent des réseaux de 18, 34, 50, 101 et 152 couches :

— Les versions **18/34 couches** utilisent un bloc résiduel « simple » à deux couches 3×3 ;

— Pour les versions plus profondes (**50/101/152**), un bloc **« bottleneck »** à trois couches (1×1, 3×3, 1×1) est utilisé : les convolutions 1×1 réduisent puis restaurent la dimensionnalité, laissant la convolution 3×3 opérer sur un espace de dimension réduite, ce qui maintient un coût de calcul comparable tout en permettant d'empiler beaucoup plus de blocs.

Remarquablement, le **ResNet-152**, bien que 8 fois plus profond que VGG-19, a une complexité (11,3 milliards de FLOPs) *inférieure* à celle de VGG-16/19 (15,3 / 19,6 milliards de FLOPs).

3.2 Détails d'entraînement (ImageNet)

— **Augmentation d'échelle** : redimensionnement avec le petit côté échantillonné aléatoirement dans $[256, 480]$, puis recadrage aléatoire 224×224 (+ retournement horizontal) ;

— **Batch Normalization** appliquée après chaque convolution et avant l'activation ; pas de *dropout* ;

— **SGD**, mini-lots de 256, taux d'apprentissage initial 0,1 divisé par 10 aux paliers, jusqu'à 60×10^4 itérations ; *weight decay* 0,0001, *momentum* 0,9.

4 Résultats expérimentaux

4.1 ImageNet — réseaux plain vs résiduels (top-1, 10-crop)

Profondeur	Plain (err. top-1 %)	ResNet (err. top-1 %)
18 couches	27,94	27,88
34 couches	28,54	25,03

TABLE 1 – Comparaison des taux d'erreur top-1 entre réseaux plain et résiduels sur ImageNet.

À 18 couches, plain et résiduel sont comparables (le problème de dégradation n'est pas encore marqué), mais le ResNet converge plus vite. À 34 couches, le renversement est net : le **ResNet-34 fait mieux que le ResNet-18** (+2,8 points), alors que le **plain-34 fait pire que le plain-18** — la dégradation est résolue par les raccourcis résiduels.

4.2 Meilleurs résultats single-model sur ImageNet (top-5)

Modèle	Top-5 (val, %)
VGG-16	9,33 (top-1 : 28,07)
GoogLeNet	9,15
PReLU-net	7,38
ResNet-50	6,71
ResNet-101	6,05
ResNet-152	5,71 (top-5 : 4,49 en full test)

TABLE 2 – Comparaison des meilleurs résultats single-model sur ImageNet.

L'ensemble de plusieurs ResNets (dont deux ResNet-152) atteint **3,57% d'erreur top-5** sur le jeu de test ImageNet, remportant la 1ère place du concours de classification ILSVRC 2015 — un résultat qui surpasse à lui seul tous les ensembles précédemment publiés, même en single-model.

4.3 CIFAR-10 : de 20 à 1202 couches

Sur CIFAR-10, les auteurs comparent des réseaux plain/résiduels de 20, 32, 44, 56 et 110 couches (identité pure, sans paramètre supplémentaire). Le meilleur résultat (**ResNet-110**, 1,7M paramètres) atteint **6,43%** d'erreur, comparable ou meilleur que des architectures beaucoup plus larges comme les *Highway Networks*.

Un réseau de **1202 couches** est aussi entraîné avec succès (erreur d'entraînement $< 0{,}1\%$), mais son erreur de test (7,93%) est légèrement supérieure à celle du ResNet-110, un effet attribué au surapprentissage sur ce petit jeu de données plutôt qu'à un problème d'optimisation.

4.4 Détection d'objets et segmentation (transfert)

En remplaçant simplement le *backbone* VGG-16 par ResNet-101 dans **Faster R-CNN**, sans autre changement, les auteurs obtiennent :

— **+3,2 points de mAP** sur PASCAL VOC 2007 (73,2 → 76,4) ;

— un **gain relatif de 28%** sur la métrique standard COCO (mAP@[.5,.95] : 21,2 → 27,2).

Ces gains, uniquement dus à la qualité des représentations apprises, ont permis à l'équipe de remporter les 1ères places en détection ImageNet, détection COCO, segmentation COCO et localisation ImageNet lors des compétitions ILSVRC & COCO 2015.

5 Analyse et discussion

— Les réponses (écart-type) de chaque couche résiduelle sont en moyenne plus faibles que celles des couches plain équivalentes, et diminuent encore avec la profondeur : cela soutient l'hypothèse que les fonctions résiduelles apprises sont en général proches de zéro, cohérent avec l'idée de préconditionnement autour de l'identité ;

— Les raccourcis « identité » (sans paramètre) sont préférés aux raccourcis « projection » : les gains de ces derniers sont marginaux, pour un coût mémoire/calcul supérieur, en particulier dans les architectures *bottleneck* où un raccourci de projection doublerait la complexité ;

— Le principe se généralise au-delà de la classification d'images : détection, localisation et segmentation en bénéficient directement, simplement en changeant le réseau *backbone*.

6 Conclusion

Ce papier introduit l'**apprentissage résiduel** comme un cadre simple, générique et quasi gratuit en calcul pour entraîner des réseaux de neurones très profonds (jusqu'à 152 couches sur ImageNet, et plus de 1000 sur CIFAR-10), en résolvant le problème de dégradation observé pour les réseaux « plain » classiques. **ResNet** est devenu depuis une architecture de référence, à la base d'un très grand nombre de modèles de vision par ordinateur ultérieurs.

7 Démonstration complémentaire (notebook)

Un notebook Jupyter joint à ce rapport (`resnet_demo.ipynb`) illustre, à très petite échelle et en NumPy pur, le principe central du papier : un perceptron multicouche « plain » profond (10 blocs) obtient une erreur d'entraînement nettement supérieure à sa version peu profonde (3

blocs) sur un jeu de données jouet, alors que l'ajout de raccourcis résiduels identiques résout ce problème de dégradation.

Il ne s'agit pas d'une reproduction des expériences ImageNet/CIFAR du papier (qui nécessiteraient un GPU et un framework de deep learning), mais d'une démonstration pédagogique du même phénomène d'optimisation, avec code et résultats exécutés.

8 Références

Références

[1] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition*. arXiv :1512.03385.

[2] Simonyan, K., & Zisserman, A. (2014). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. arXiv :1409.1556.

[3] Szegedy, C., Liu, W., Jia, Y., et al. (2015). *Going Deeper with Convolutions*. CVPR 2015.

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving Deep into Rectifiers : Surpassing Human-Level Performance on ImageNet Classification*. ICCV 2015.

[5] Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN : Towards Real-Time Object Detection with Region Proposal Networks*. NeurIPS 2015.

[6] Russakovsky, O., Deng, J., Su, H., et al. (2015). *ImageNet Large Scale Visual Recognition Challenge*. International Journal of Computer Vision, 115(3), 211–252.

[7] Lin, T.Y., Maire, M., Belongie, S., et al. (2014). *Microsoft COCO : Common Objects in Context*. ECCV 2014.

[8] Srivastava, R.K., Greff, K., & Schmidhuber, J. (2015). *Highway Networks*. arXiv :1505.00387.